Flask Framework

Study notes - Section 4 (Lectures 44–49) - Complete MLOps Bootcamp

1. Introduction to Flask (Lecture 44)

Flask is a web framework created with the Python programming language. It is a *micro-framework*: the core is small (routing, requests/responses, templates) and everything else (databases, auth, forms) is added through extensions. That makes it a popular choice for quickly serving ML models as web apps or REST APIs.

Flask stands on two pillars:

(1) WSGI — Web Server Gateway Interface

WSGI is the standard (PEP 3333) that defines how a web server talks to a Python web application. The server receives the HTTP request, hands it to the app through the WSGI interface, and sends the app's response back to the client. Flask implements WSGI using the **Werkzeug** toolkit.

```
Browser / Client  --HTTP request-->  Web server (Gunicorn, uWSGI, ...)
                                          |  WSGI interface
                                          v
                                     Flask app  (your Python code)
                                          |
Browser / Client  <--HTTP response--  Web server  <-------+
```

Development vs production: `app.run()` starts Werkzeug's built-in development server, which is not meant for production. In production you run Flask behind a WSGI server such as Gunicorn, e.g. `gunicorn -w 4 app:app`.

(2) Jinja2 — Template Engine

Jinja2 lets you write HTML files with placeholders and logic that Flask fills with Python data before sending the page to the browser. This turns static HTML into dynamic pages (covered in Lecture 48).

Installation

```
pip install flask          # or, with uv:  uv add flask
python -c "import flask; print(flask.__version__)"
```

2. Understanding a Sample Flask Application (Lecture 45)

app.py

```
from flask import Flask

# Create an instance of the Flask class: this is the WSGI application.
# __name__ tells Flask where to find templates and static files.
app = Flask(__name__)

@app.route("/")              # decorator: map the URL "/" to this function
def welcome():
    return "Welcome to this Flask course. This should be an amazing course"

@app.route("/index")
def index():
    return "Welcome to the index page"

if __name__ == "__main__":
    app.run(debug=True)      # debug=True -> auto-reload + interactive error pages
```

- **app = Flask(__name__)** creates the application object.
- **@app.route(path)** registers a *view function*; whatever it returns becomes the HTTP response.
- **debug=True** restarts the server when you save a file, so you don't have to stop/start it manually. Never use it in production.
- Run with `python app.py` and open `http://127.0.0.1:5000` (Flask's default port is 5000).

3. Integrating HTML with Flask (Lecture 46)

Instead of returning strings, return full HTML pages with **render_template**. Flask looks for them in a folder named **templates/** next to your app file (the name is mandatory).

```
project/
|-- main.py
`-- templates/
    |-- index.html
    |-- about.html
    `-- form.html
```

main.py

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def welcome():
    return "<html><h1>Welcome to the Flask course</h1></html>"   # inline HTML works too

@app.route("/index")
def index():
    return render_template("index.html")

@app.route("/about")
def about():
    return render_template("about.html")

if __name__ == "__main__":
    app.run(debug=True)
```

templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Home</title></head>
<body>
  <h1>Welcome to the Home Page</h1>
  <p>This page is rendered by Flask with render_template().</p>
</body>
</html>
```

4. HTTP Verbs: GET and POST (Lecture 47)

Verb	Purpose	Typical use
GET	Request / read data from the server	Open a page, fetch a resource
POST	Send data to the server to create something	Submit a form, call a prediction endpoint
PUT	Update / replace an existing resource	Edit an item by id
DELETE	Remove a resource	Delete an item by id

By default a route only accepts GET. Use the **methods** argument to accept more, and the **request** object to read what the client sent.

getpost.py

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/form", methods=["GET", "POST"])
def form():
    if request.method == "POST":
        name = request.form["name"]          # value of <input name="name">
        return f"Hello {name}!"
    return render_template("form.html")      # GET -> just show the form
```

templates/form.html

```
<form action="/form" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>
  <input type="submit" value="Submit">
</form>
```

- **request.form** – data from an HTML form (POST).
- **request.args** – query-string parameters, e.g. `/search?q=flask` (GET).
- **request.json** / **request.get_json()** – JSON body sent by an API client.

5. Dynamic URLs with Jinja2 (Lecture 48)

Variable rules

Parts of the URL can be variables, optionally with a converter that also validates the type: `<string:x>` (default), `<int:x>`, `<float:x>`, `<path:x>`.

```
@app.route("/success/<int:score>")
def success(score):
    res = "PASS" if score >= 50 else "FAIL"
    return render_template("result.html", results=res, score=score)
```

Jinja2 syntax

Syntax	Meaning
`{{ variable }}`	Print an expression / variable into the HTML
`{% ... %}`	Statements: `if`, `for`, `block`, `extends`
`{# ... #}`	Comments (not sent to the browser)

templates/result.html

```
<h2>Final result</h2>
<p>Your score is {{ score }}</p>

{% if score >= 50 %}
  <p style="color:green">You passed!</p>
{% else %}
  <p style="color:red">You failed.</p>
{% endif %}

<ul>
{% for key, value in results.items() %}
  <li>{{ key }}: {{ value }}</li>
```

```
{% endfor %}
</ul>
{# results must be a dict for the loop above #}
```

Form → compute → redirect to a dynamic URL

jinja.py

```
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

@app.route("/submit", methods=["GET", "POST"])
def submit():
    total_score = 0
    if request.method == "POST":
        science = float(request.form["science"])
        maths   = float(request.form["maths"])
        c       = float(request.form["c"])
        data_sc = float(request.form["datascience"])
        total_score = (science + maths + c + data_sc) / 4
        # url_for builds the URL from the *function name*, not the path
        return redirect(url_for("successres", score=int(total_score)))
    return render_template("getresult.html")

@app.route("/successres/<int:score>")
def successres(score):
    res = "PASS" if score >= 50 else "FAIL"
    exp = {"score": score, "res": res}
    return render_template("result1.html", results=exp)
```

Why url_for? If you later rename a route's path, every `url_for('successres', ...)` keeps working because it references the function name. It also escapes special characters for you.

6. PUT and DELETE: Building a REST API (Lecture 49)

For APIs we return JSON instead of HTML using **jsonify**. The example below implements full CRUD over an in-memory list (a real app would use a database).

api.py

```
from flask import Flask, jsonify, request

app = Flask(__name__)

items = [
    {"id": 1, "name": "Item 1", "description": "This is item 1"},
    {"id": 2, "name": "Item 2", "description": "This is item 2"},
]

@app.route("/")
def home():
    return "Welcome to the sample To Do List App"

# GET: retrieve all items
@app.route("/items", methods=["GET"])
def get_items():
    return jsonify(items)

# GET: retrieve one item by id
@app.route("/items/<int:item_id>", methods=["GET"])
def get_item(item_id):
    item = next((i for i in items if i["id"] == item_id), None)
    if item is None:
        return jsonify({"error": "item not found"}), 404
```

```
    return jsonify(item)

# POST: create a new item
@app.route("/items", methods=["POST"])
def create_item():
    if not request.json or "name" not in request.json:
        return jsonify({"error": "name is required"}), 400
    new_item = {
        "id": items[-1]["id"] + 1 if items else 1,
        "name": request.json["name"],
        "description": request.json.get("description", ""),
    }
    items.append(new_item)
    return jsonify(new_item), 201

# PUT: update an existing item
@app.route("/items/<int:item_id>", methods=["PUT"])
def update_item(item_id):
    item = next((i for i in items if i["id"] == item_id), None)
    if item is None:
        return jsonify({"error": "item not found"}), 404
    item["name"] = request.json.get("name", item["name"])
    item["description"] = request.json.get("description", item["description"])
    return jsonify(item)

# DELETE: remove an item
@app.route("/items/<int:item_id>", methods=["DELETE"])
def delete_item(item_id):
    global items
    items = [i for i in items if i["id"] != item_id]
    return jsonify({"result": "item deleted"})

if __name__ == "__main__":
    app.run(debug=True)
```

Testing the API

A browser can only send GET easily, so use Postman or curl for the other verbs:

```
curl http://127.0.0.1:5000/items
curl -X POST   -H "Content-Type: application/json" \
     -d '{"name": "Item 3", "description": "new"}'  http://127.0.0.1:5000/items
curl -X PUT    -H "Content-Type: application/json" \
     -d '{"name": "Updated item"}'                  http://127.0.0.1:5000/items/1
curl -X DELETE http://127.0.0.1:5000/items/1
```

Status code	Meaning
200 OK	Request succeeded (default)
201 Created	A new resource was created (POST)
400 Bad Request	Client sent invalid / incomplete data
404 Not Found	The requested resource does not exist
500 Internal Server Error	Unhandled exception in your code

7. Cheat Sheet

Task	Code
Create app	`app = Flask(__name__)`
Define route	`@app.route('/path', methods=['GET','POST'])`

Render HTML	`render_template('page.html', var=value)`
Read form data	`request.form['field']`
Read query params	`request.args.get('q')`
Read JSON body	`request.get_json()`
Return JSON + status	`return jsonify(obj), 201`
Dynamic URL	`@app.route('/user/<int:id>')`
Redirect	`redirect(url_for('func_name', arg=x))`
Run (dev)	`app.run(debug=True)` or `flask --app app run --debug`
Run (prod)	`gunicorn -w 4 app:app`

Flask vs FastAPI at a glance

Aspect	Flask	FastAPI
Interface	WSGI (synchronous)	ASGI (async-native)
Data validation	Manual (or extensions)	Automatic with Pydantic + type hints
API docs	Not built in	Auto-generated Swagger UI at /docs
Templates	Jinja2 built in	Jinja2 supported, mostly used for APIs
Route decorator	`@app.route('/x', methods=['POST'])`	`@app.post('/x')`
Prod server	Gunicorn / uWSGI	Uvicorn (or Gunicorn + Uvicorn workers)